Unir CP2

Despliegue y configuración de infraestructura en Azure

Carlos Grande

Copyright © Unir CP2 - 2025

Table of contents

1. Inicio 3
1.1 Introducción 3
1.2 Objetivos 3
2. Informe 4
2.1 Informe 4
2.2 Arquitectura 5
2.3 Despliegue 10
2.4 Evidencias 15
2.5 Licencia 16
2.6 Referencias 17

Copyright © Unir CP2 - 2025

1. Inicio

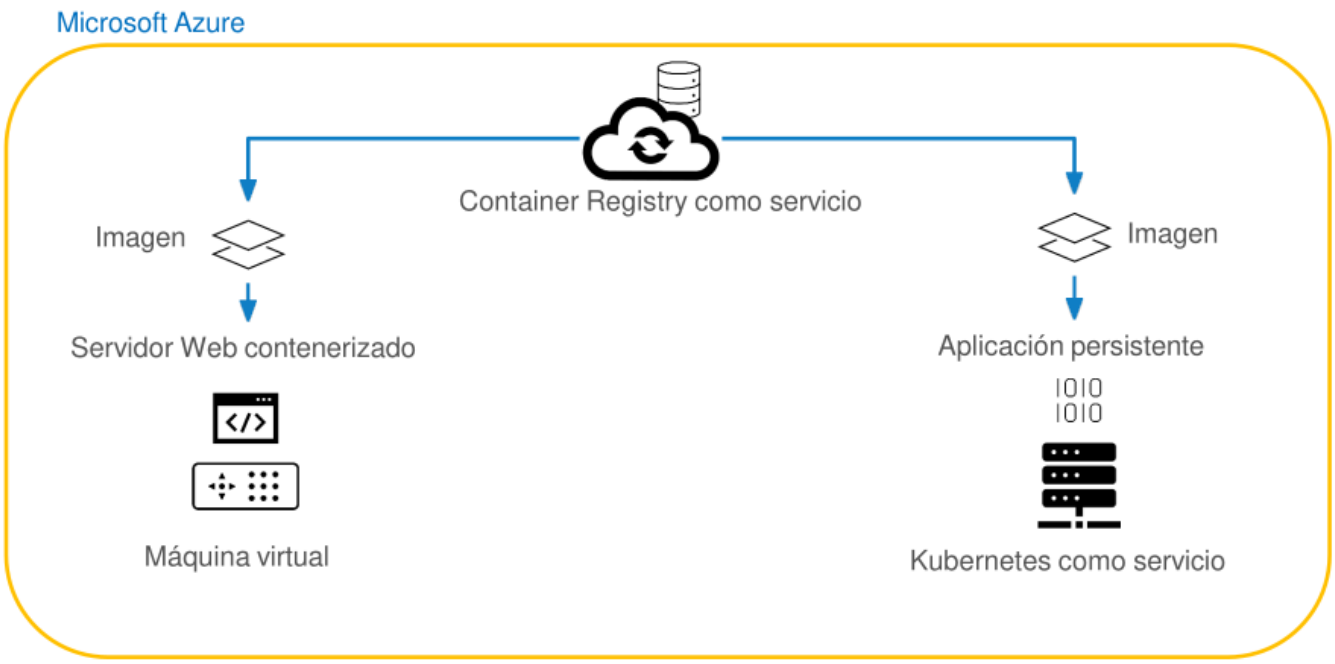
Asignatura: **Devops & Cloud**

Alumno: **Carlos Grande Núñez**

Fecha: **23/03/25**

1.1 Introducción

Este repositorio contiene la solución del **Caso Práctico 2**, en el cual se ha desplegado una infraestructura en **Microsoft Azure** de forma automatizada utilizando **Terraform** y **Ansible**. Se incluyen configuraciones para la creación de recursos en la nube, instalación de servicios y despliegue de aplicaciones en contenedores con almacenamiento persistente.



1.2 Objetivos

- Crear infraestructura de forma automatizada en un proveedor de Cloud pública.
- Utilizar herramientas de gestión de la configuración para automatizar la instalación y configuración de servicios.
- Desplegar mediante un enfoque totalmente automatizado aplicaciones en forma de contenedor sobre el sistema operativo.
- Desplegar mediante un enfoque totalmente automatizado aplicaciones que hagan uso de almacenamiento persistente sobre una plataforma de orquestación de contenedores.

Copyright © Unir CP2 - 2025

2. Informe

2.1 Informe

2.1.1 Codigo fuente

Acceso al repositorio

2.1.2 🗂️ Estructura del repositorio

```
📦 repo-root
├── ansible
│   ├── deploy.sh       # Script de despliegue con Ansible
│   ├── hosts           # Inventario de servidores
│   └── playbook.yml    # Playbook principal de Ansible
│
├── terraform
│   ├── vars.tf         # Variables de configuración
│   ├── main.tf         # Configuración principal de Terraform
│   └── recursos.tf     # Definición de recursos en Azure
```


Copyright © Unir CP2 - 2025

2.2 Arquitectura

Esta sección describe la infraestructura desplegada en Azure, estructurado de la siguiente manera:

- Diagrama general
- Infraestrutura
- Configuración de la infraestructura

2.2.1 Diagrama general

El siguiente diagrama representa la infraestructura desplegada con Terraform y configurada con Ansible, incluyendo una máquina virtual con un contenedor Podman y un clúster AKS, ambos obteniendo imágenes desde un Azure Container Registry (ACR).

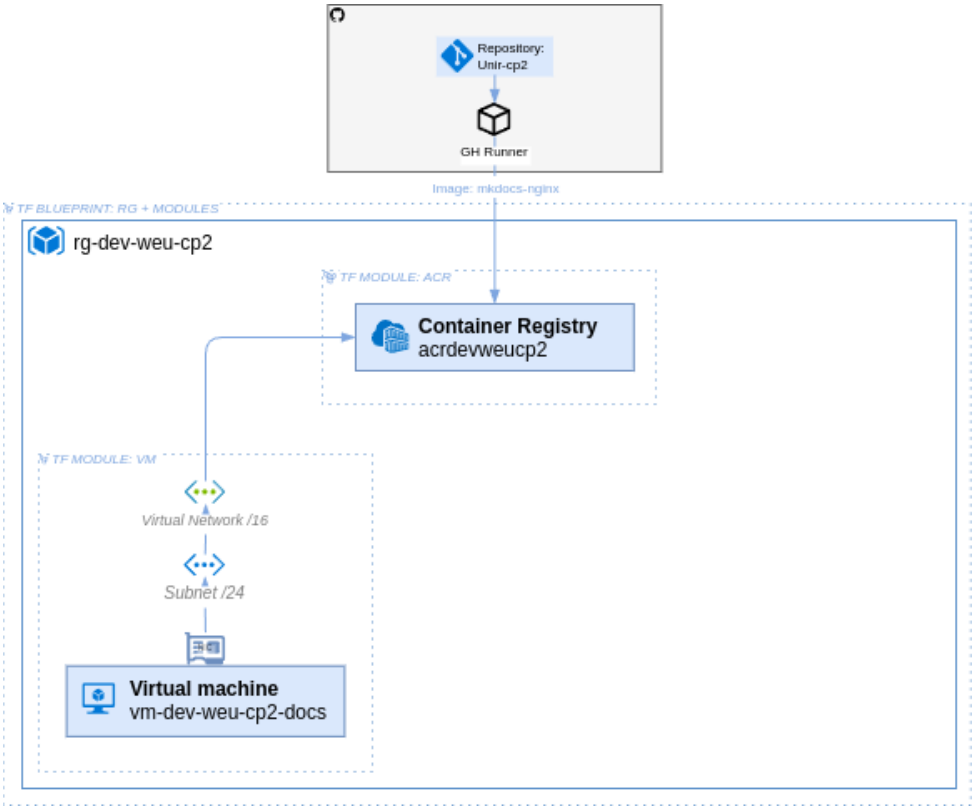


Figura 1: Diagrama de la arquitectura desplegada en Azure (Elaboración propia con draw.io).

2.2.2 Infraestructura

A continuación se describen los diferentes componentes de la infraestructura desplegados con terraform y la justificación de sus configuraciones.

Copyright © Unir CP2 - 2025

Container registry

Máquina virtual

La infraestructura de la máquina virtual se ha definido utilizando **Terraform**, organizando los recursos en módulos separados para mejorar la modularidad y reutilización del código. A continuación, se presentan los archivos principales que definen el despliegue:

```
terraform/
|── terraform.tfvars        # Variables globales del despliegue
|── main.tf                 # Llamada a módulos y recursos principales
|── modules/
|   ├── vm/                 # Módulo de la máquina virtual
|   |   ├── main.tf         # Definición de la VM
|   |   ├── network.tf      # Configuración de la red
|   |   ├── security.tf     # Reglas de seguridad (NSG)
|   |   ├── outputs.tf      # Variables de salida (IPs, VM ID)
|   |   └── variables.tf    # Definición de variables del módulo
```

DEFINICIÓN DE LA MÁQUINA VIRTUAL

La máquina virtual está configurada en **Azure** y la infraestructura de la máquina virtual se ha organizado en un módulo dentro de **Terraform**

main.tf

```tf
# Public IP
resource "azurerm_public_ip" "vm_public_ip" {
  name                = "${var.vm_name}-public-ip"
  resource_group_name = var.resource_group
  location            = var.location
  allocation_method   = "Static"
}

# Network Interface
resource "azurerm_network_interface" "nic" {
  name                = "${var.vm_name}-nic"
  resource_group_name = var.resource_group
  location            = var.location

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    public_ip_address_id          = azurerm_public_ip.vm_public_ip.id
    private_ip_address_allocation = "Dynamic"
  }
}

# Virtual Machine
resource "azurerm_linux_virtual_machine" "vm" {
  name                  = var.vm_name
  resource_group_name   = var.resource_group
  location              = var.location
  size                  = var.vm_size
  admin_username        = var.admin_username
  network_interface_ids = [azurerm_network_interface.nic.id]

  admin_ssh_key {
    username   = var.admin_username
    public_key = var.ssh_public_key
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = var.image_offer
    sku       = var.image_os
    version   = "latest"
  }
}
```

- Se ha elegido **Ubuntu 22.04 LTS (Gen2)** como sistema operativo, basado en la imagen oficial de **Canonical**.
- La máquina está configurada para autenticación por **clave SSH**, evitando contraseñas.
- Se asigna una **interfaz de red**, conectada a una **subred dentro de una VNet**.
- Se utiliza un disco gestionado con almacenamiento **Standard_LRS** para optimizar costes.

Copyright © Unir CP2 - 2025

REDES DE LA MÁQUINA VIRTUAL

La red virtual y la subred asociada a la máquina virtual están definidas en `network.tf`, garantizando su conectividad.

network.tf

```
# Definición de la Virtual Network (VNet)
resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  resource_group_name = var.resource_group
  location            = var.location
  address_space       = ["10.0.0.0/16"]
}

# Subred dentro de la VNet
resource "azurerm_subnet" "subnet" {
  name                 = var.subnet_name
  resource_group_name  = var.resource_group
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = [var.subnet_cidr]
}

# Interfaz de red de la VM
resource "azurerm_network_interface" "nic" {
  name                = "${var.vm_name}-nic"
  resource_group_name = var.resource_group
  location            = var.location

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    public_ip_address_id          = azurerm_public_ip.vm_public_ip.id
    private_ip_address_allocation = "Dynamic"
  }
}

# Dirección IP pública de la VM
resource "azurerm_public_ip" "vm_public_ip" {
  name                = "${var.vm_name}-public-ip"
  resource_group_name = var.resource_group
  location            = var.location
  allocation_method   = "Static"
}
```

- Se define una **VNet (`vnet-weu-cp2`)** con un **espacio de direcciones** `10.0.0.0/16`.
- La **subred (`subnet-weu-cp2`)** tiene un rango más reducido `10.0.1.0/28` para optimizar la asignación de IPs.
- La VM obtiene una **IP pública estática**, permitiendo acceso remoto controlado.

SEGURIDAD DE LA MÁQUINA VIRTUAL

El **grupo de seguridad (NSG)** controla el tráfico de entrada y salida de la VM. En `security.tf`, se han definido reglas explícitas para habilitar el acceso SSH, HTTP y HTTPS.

security.tf

```
# Definición del Network Security Group (NSG)
resource "azurerm_network_security_group" "vm_nsg" {
  name                = "${var.vm_name}-nsg"
  resource_group_name = var.resource_group
  location            = var.location
}

# Regla para permitir SSH (puerto 22)
resource "azurerm_network_security_rule" "allow_ssh" {
  name                        = "Allow-SSH"
  priority                    = 1000
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "22"
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  network_security_group_name = azurerm_network_security_group.vm_nsg.name
}

# Regla para permitir HTTP (puerto 80)
resource "azurerm_network_security_rule" "allow_http" {
  name                        = "Allow-HTTP"
  priority                    = 1010
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "80"
```


Copyright © Unir CP2 - 2025

```
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  network_security_group_name = azurerm_network_security_group.vm_nsg.name
}

# Regla para permitir HTTPS (puerto 443)
resource "azurerm_network_security_rule" "allow_https" {
  name                        = "Allow-HTTPS"
  priority                    = 1020
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "443"
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  network_security_group_name = azurerm_network_security_group.vm_nsg.name
}

# Regla para permitir todo el tráfico de salida
resource "azurerm_network_security_rule" "allow_outbound" {
  name                        = "Allow-All-Outbound"
  priority                    = 900
  direction                   = "Outbound"
  access                      = "Allow"
  protocol                    = "*"
  source_port_range           = "*"
  destination_port_range      = "*"
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  network_security_group_name = azurerm_network_security_group.vm_nsg.name
}
```

- Se permite acceso **SSH (22) solo a usuarios autenticados**.
- Se abren los **puertos HTTP (80) y HTTPS (443)** para servir contenido web.
- Se garantiza que la VM pueda **salir a internet sin restricciones**, útil para actualizaciones o conexión a otros servicios.

Kubernetes service

Imágenes contenerizadas

IMÁGEN SIN PERSISTENCIA PARA LA VM

La imagen utilizada en el contenedor Podman dentro de la máquina virtual se basa en **MkDocs**, una librería de documentación escrita en Python. Esta herramienta permite generar sitios estáticos a partir de archivos Markdown, facilitando la creación y publicación de documentación técnica (MkDocs, s.f.). La imagen generada en este ejercicio contiene la documentación del propio proyecto, asegurando que el contenido se pueda visualizar de manera estructurada en un navegador.

Además, se ha utilizado el tema **Material for MkDocs**, que añade una interfaz moderna y varias opciones de personalización (Squidfunk, s.f.).

Publicación en GitHub Pages

La documentación también está disponible a través de **GitHub Pages**, lo que permite su acceso incluso cuando la infraestructura de Azure no está desplegada. Se puede visualizar en el siguiente enlace:

Ver documentación en GitHub Pages

Generación de la Imagen

La imagen se genera a partir de la documentación escrita en MkDocs, transformándola en un sitio web estático y empaquetándola en un contenedor. Esta imagen se construye y publica mediante dos métodos:

1. **Workflow de GitHub:** Se ha añadido un workflow en `.github/workflows` llamado `Publish docs to ACR`, que permite generar y publicar la imagen en el ACR.
2. **Ejecución con Ansible:** Durante la configuración de la máquina virtual, Ansible ejecuta un playbook con el mismo proceso que el workflow de GitHub para generar y publicar la imagen en el ACR.

El proceso detallado de despliegue de la imagen puede consultarse en el siguiente apartado de esta memoria: Sección de Despliegue.

Copyright © Unir CP2 - 2025

IMÁGEN CON PERSISTENCIA PARA EL AKS

2.2.3 Configuración de la infraestructura

A continuación se describen las configuraciones aplicadas a la infraestructura desplegada, realizadas con Ansible, y la justificación de cada una de ellas.

Configuración de la VM

Configuración del AKS

Copyright © Unir CP2 - 2025

2.3 Despliegue

A continuación, se explica cómo reproducir los pasos necesarios para llevar a cabo el caso práctico sobre el repositorio. Se detallan las instrucciones para:

- 1. Despliegue de la infraestructura
- 2. Publicación de las imagenes
- 3. Configuración de la VM
- 4. Configuración del AKS

2.3.1 1. Despliegue de la infraestructura

El despliegue de la infraestructura se realiza con Terraform desde la máquina local, asegurando que la configuración es válida antes de aplicar los cambios y provisionar los recursos necesarios.

1. Accede al directorio de terraform en el repositorio e inicializa terraform.

```
cd terraform
terraform init
```

Output: `Terraform has been successfully initialized!`

2. Ejecuta la validación de los ficheros generados con el siguiente comando:

```
terraform validate
```

output: `Success! The configuration is valid.`

3. Despliega la infraestructura con el siguiente comando, por defecto se despliega en dev. Siempre puedes añadir el flag `-var="environment=pro"` para especificar un entorno entre `dev|pre|pro`

```
terraform apply --auto-approve
cd ..
```

Automatización de variables

Tras el despliegue de toda la infraestructura se generan automáticamente las variables globales necesarias para poder realizar lo que queda del ejercicio ejecutando el fichero `setup.sh`.

```
source setup.sh
```


Copyright © Unir CP2 - 2025

2.3.2 2. Publicación de las imagenes

La publicación de la imagen se automatiza mediante el workflow `Publish release to ACR` de GitHub Actions, que envía la imagen al Azure Container Registry (ACR). Para ello, se deben proporcionar las credenciales adecuadas y validar la ejecución del proceso.

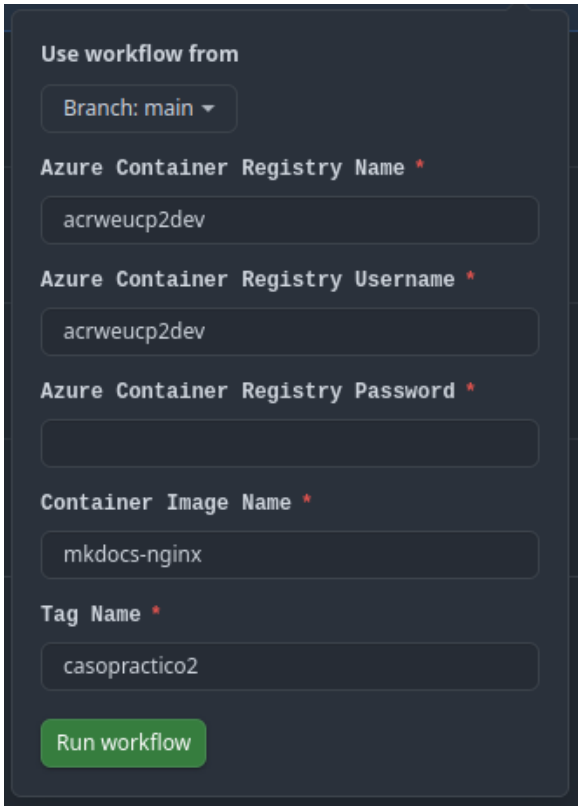
Copyright © Unir CP2 - 2025

1. Rellenar los datos del formulario del workflow con username y pwd del ACR desplegado en Azure.

Visualizar usuario y contraseña del ACR

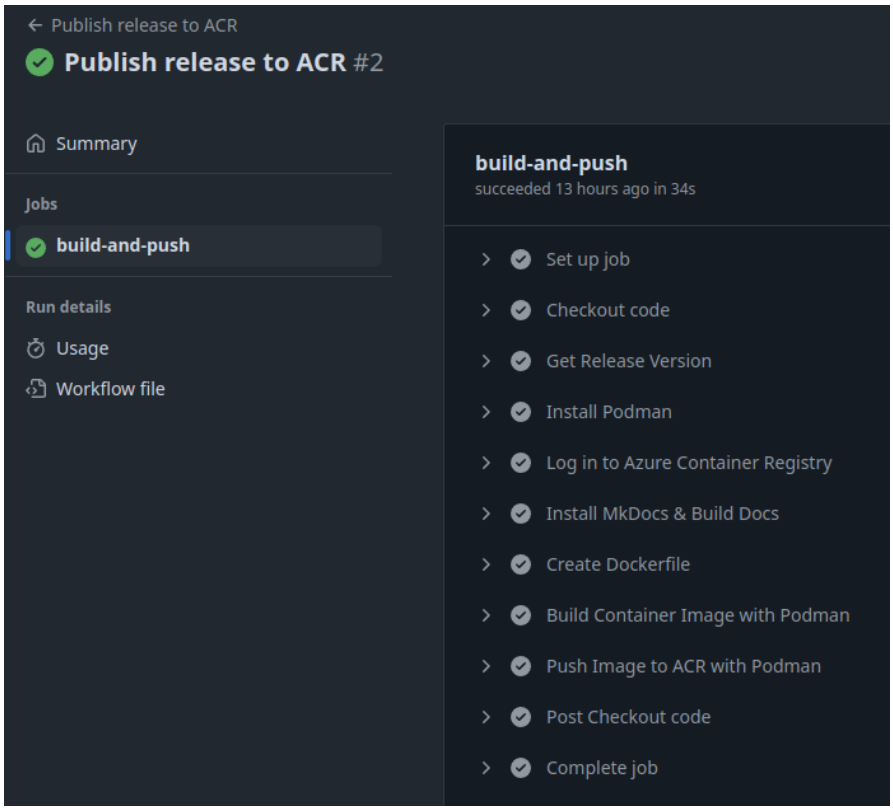
Siempre puedes ejecutar este comando para recuperar el usuario y la contraseña del ACR.

```
az acr credential show --name acrweucp2dev --query "[username, passwords[0].value]" -o tsv
```



2. Ejecutar workflow y validar la correcta ejecución del job

Copyright © Unir CP2 - 2025



2.3.3 3. Configuración de la VM

La configuración de la VM se llevará a cabo desde la máquina local utilizando Ansible, accediendo por SSH para realizar comprobaciones y garantizar el correcto despliegue del entorno.

Copyright © Unir CP2 - 2025

1. Comprobar conexión a la VM por SSH

```
ssh -i ~/.ssh/az_unir_rsa charlstown@${VM_IP}
exit
```

2. Ejecutar ansible apuntando a la VM. Asegurarse que el comando se ejecuta desde `./ansible`. Para forzar ansible a recrear todo desde el principio es posible usar los argumentos `--force-handlers` y `--extra-vars "recreate=true"`.

> **Fichero de configuración de Ansible**
>
> Según la documentación de Ansible, es importante declarar la ubicación del fichero de configuración si este se encuentra en el mismo directorio desde el que se ejecuta (Ansible, s.f.).
>
> ```
> export ANSIBLE_CONFIG=./ansible.cfg
> ```

```
ansible-playbook -i inventory.ini install_podman_run_container.yml --extra-vars "@vars.yml" --extra-vars "ansible_host=$VM_IP" --ask-vault-pass
```

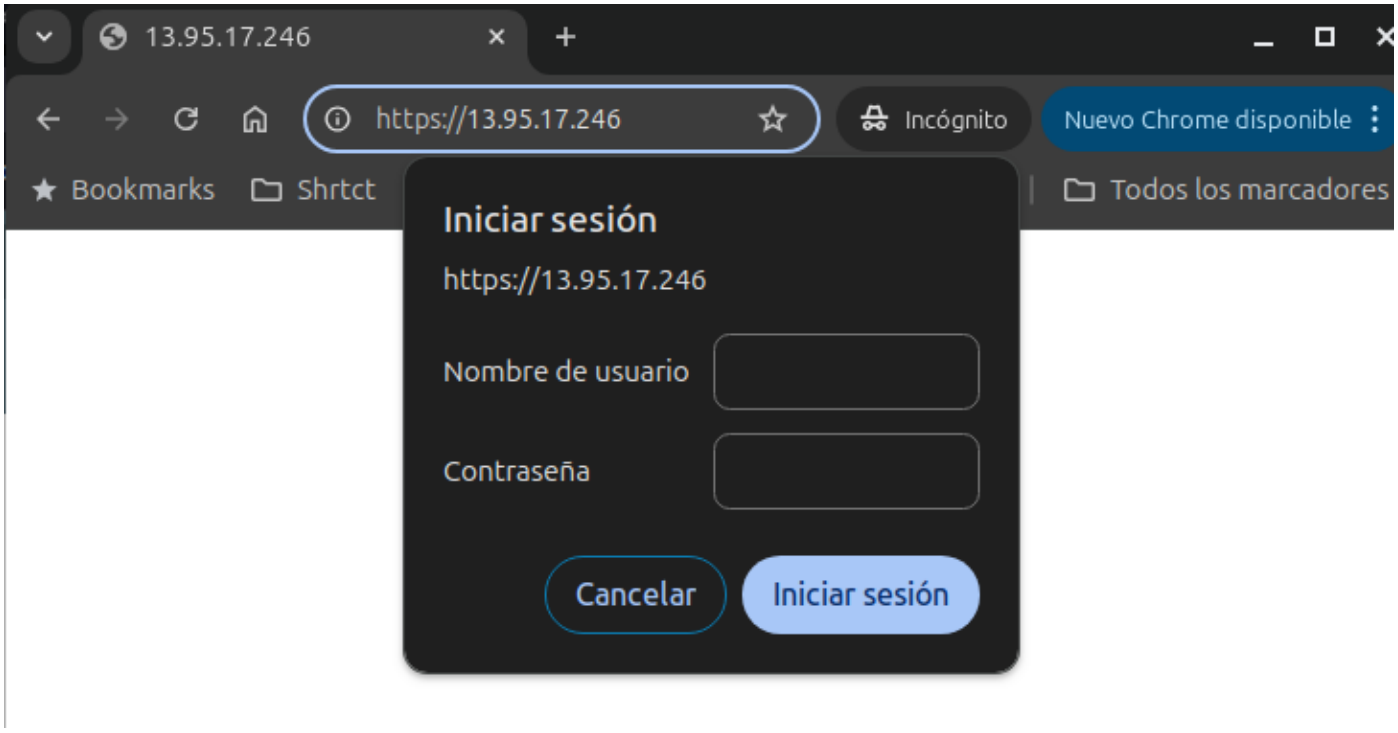
Este playbook se ejecuta apuntando a un Vault de ansible donde se han guardado las credenciales usadas para crear el fichero `htpasswd.users` en la carpeta `/etc/nginx/auth/htpasswd.users` de la VM.

> **Mostrar contraseñas guardadas en el vault**
>
> Para visualizar las contraseñas guardadas en el vault puedes ejecutar el comando:
>
> ```
> ansible-vault view secrets.yml
> ```

3. Si todo ha ido bien se puede comprobar que el sitio se muestra a través de internet en la ip pública de la VM. Ejecutando el comando:

```
curl -k -u charlstown:\*\*\* https://${VM_IP}:443
```

También puede visualizarse en el browser en la dirección `https://ip-publica/`.



2.3.4 4. Configuración del AKS

Copyright © Unir CP2 - 2025

2.4 Evidencias

A continuación, se exponen las evidencias de los procesos empleados para realizar la práctica.

- 1. Despliegue de la infraestructura
- 2. Publicación de las imagenes
- 3. Configuración de la VM
- 4. Configuración del AKS

2.4.1 1. Despliegue de la infraestructura

2.4.2 2. Publicación de las imagenes

2.4.3 3. Configuración de la VM

2.4.4 4. Configuración del AKS

Copyright © Unir CP2 - 2025

2.5 Licencia

Copyright © Unir CP2 - 2025

2.6 Referencias

Ansible. (s.f.). *Ansible configuration settings*. Ansible Documentation. Recuperado de https://docs.ansible.com/ansible/latest/reference_appendices/config.html

2.6.1 Herramientas usadas

- JGraph Ltd. (s.f.). *draw.io*. Recuperado de https://www.drawio.com/
- MkDocs. (s.f.). *MkDocs Documentation*. Recuperado de https://www.mkdocs.org
- Squidfunk. (s.f.). *Material for MkDocs*. Recuperado de https://squidfunk.github.io/mkdocs-material/

Copyright © Unir CP2 - 2025